Got it — this is a **compiler-focused comparison from Java 1.0 → Java 21**, not language features in general. Below is a **clean, interview-ready table** showing **similarities and dissimilarities of Java compilers (javac)** across versions.

▯ Similarities Across All Java Compilers (Java 1 → 21)

These characteristics **never changed**, regardless of version.

Aspect	Java 1 – Java 21 (Same Behavior)
Compiler Name	Javac
Input	.java source files
Output	.class bytecode files
Platform Independence	Bytecode runs on any JVM
Compilation Type	Ahead-of-Time (AOT)
Syntax Checking	Compile-time syntax validation
Type Checking	Strong static typing
Bytecode Verification	JVM verifies bytecode
Error Reporting	Line number + message
Symbol Resolution	Compile-time linking
Access Control Checks	public/private/protected
Exception Checking	Checked exceptions enforced
Classpath Resolution	-classpath / -cp
Backward Compatibility	New compiler can compile old code
JVM Targeting	-source and -target support
No Runtime Execution	Compiler does not execute code

 Dissimilarities Between Java Compilers (Version-Wise)

 Core Compiler Architecture Evolution

Feature	Java 1–4	Java 5–7	Java 8	Java 9–11	Java 12–16	Java 17–21
Compiler Phases	Basic	Improved	Lambda aware	Module aware	Preview aware	Pattern aware
AST Complexity	Simple	Moderate	Complex	Highly complex	Advanced	Very advanced
Bytecode Generation	Basic	Enhanced	invokedynamic	Optimized	Optimized	Optimized
Internal APIs	Few	Growing	LambdaMetafactory	Module APIs	Sealed APIs	Record APIs

---

 Language Feature Compilation Support

Feature	Java Version Introduced	Compiler Change
Inner Classes	Java 1.1	Synthetic class generation
Generics	Java 5	Type erasure logic
Annotations	Java 5	Metadata processing
Enums	Java 5	Special bytecode
Varargs	Java 5	Array synthesis
Diamond Operator	Java 7	Type inference
Lambda Expressions	Java 8	Invokedynamic
Method References	Java 8	Lambda linkage
Default Methods	Java 8	Interface bytecode changes
Modules	Java 9	Module graph resolution
var keyword	Java 10	Local type inference
Switch Expressions	Java 12+	New control flow bytecode

Text Blocks	Java 13+	String literal handling
Records	Java 16	Canonical constructor gen
Sealed Classes	Java 17	Inheritance validation
Pattern Matching	Java 16–21	Flow-sensitive typing
Virtual Threads	Java 21	No compiler impact (runtime)

□ Bytecode & JVM Interaction Differences

Aspect	Java 1–4	Java 5–7	Java 8	Java 9–21
Bytecode Version	45–48	49–51	52	53–65
StackMap Frames	✕ No	✔Partial	✔Mandatory	✔Mandatory
invokedynamic	✕ No	✕ No	✔Yes	✔Optimized
Constant Pool	Small	Medium	Large	Very large
Metadata Storage	PermGen	PermGen	Metaspace	Metaspace

□ Module System Impact

Aspect	Pre-Java 9	Java 9+
Module Awareness	✕ No	✔Yes
module-info.java	✕ No	✔Compiled
Strong Encapsulation	✕ No	✔Enforced
Split Packages	Allowed	Compilation Error
Illegal Access	Allowed	Warning/Error

□ Annotation Processing Evolution

Feature	Java 5	Java 6–7	Java 8+
Basic Annotations	✔	✔	✔
Annotation Processor API	✕	✔	✔
Compile-time Code Generation	✕	✔	✔
Repeatable Annotations	✕	✕	✔

□ Compiler Flags & Tooling

Capability	Java 1–4	Java 5–7	Java 8	Java 9–21
-source	✕	✔	✔	✔
-target	✕	✔	✔	✔
--release	✕	✕	✕	✔
Preview Features	✕	✕	✕	✔
-Xlint	✕	✔	✔	✔
Warning Levels	Basic	Moderate	Advanced	Very Advanced

□ Performance & Optimization Differences

Aspect	Java 1–4	Java 5–7	Java 8	Java 9–21
Compilation Speed	Slow	Medium	Faster	Fastest
Incremental Compilation	✕	✕	Partial	✔
Memory Usage	High	High	Medium	Optimized

Error Diagnostics	Poor	Better	Good	Excellent

 Error Diagnostics & Developer Experience

Feature	**Java 1–4**	**Java 5–7**	**Java 8**	**Java 11+**
Error Messages	Cryptic	Improved	Clear	Human-friendly
Type Inference Errors	✕	✕	✔	✔✔
Suggestive Errors	✕	✕	✕	✔
Nullability Help	✕	✕	✕	✔(patterns)

 One-Line Summary

All Java compilers from 1 to 21 share the same fundamental job — converting Java source to bytecode — but evolve dramatically in intelligence, modularity, diagnostics, bytecode sophistication, and language awareness.

If you want next:

- **Java 1–21 compiler changes interview Q&A**
- **How javac works internally (phases diagram)**
- **Why compiler ≠ JIT (deep explanation)**
- **Java compiler vs Kotlin vs Scala**